

Proofs and Prices: Machine-Checked Soundness and Measured Trade-offs in a Minimalist Chess Engine

Thomas Dybdahl Ahle Second Author

Draft of August 6, 2026

Abstract

Sunfish is a chess engine whose whole logic fits in 131 lines of Python, and whose docstrings state, in unusually precise English, the contracts its search is supposed to satisfy. We report on a one-day campaign that treated those docstrings as latent formal specifications: we transcribed them into Lean 4, proved the ones that are true, refuted the ones that are not, and named the ones that are only conditionally true — and, in parallel, priced every resulting engine change with ELO tournaments under a fixed empirical methodology. The formal deliverables include a sorry-free proof of the engine’s core fail-soft null-window contract, a machine-checked seven-position counterexample showing the engine’s stalemate correction is sound only under a named hypothesis, a same-day proof of a maintainer-conjectured invariant about killer moves, and a machine-checked transposition-table inconsistency induced by gamma-adaptive Late Move Reductions. For the latter we first proved an interval-shaped statement — fail highs bound one value function, fail lows another, the truth in between — and then deleted it: such a claim is a guarantee only if the gap between the two functions is bounded, and no bound on search instability is provable (one ply of depth can hide a mate), so the interval reading was never a specification, only a description of a bug. The only contract the engine recognizes is the point spec: every stored bound describes one value function determined by the transposition key. These results organize into a soundness taxonomy for selective search: the search window may shape termination, and may trigger shortcuts whose value provably one-side-bounds the same value function; it must never select between incomparable evaluations. Each unproven assumption is kept as a named hypothesis — a ledger of bets — and, where possible, measured on the production engine, which plays publicly on Lichess. The story ends with a measurement: an honest variant of the reductions — identical cutoffs, sound bound propagation — turned out to be worth exactly 0.00 ELO, so the mechanism’s entire measured value was its unsoundness, and the engine now ships with no reductions at all — full-width, and provable exactly as its docstring is written. Proofs identify the trade; tournaments price it.

1 Introduction

Sunfish¹ is a minimalist chess engine: with comments and whitespace removed, the engine proper is 131 lines of Python, yet it plays at ratings above 2000 on Lichess, where it runs in production as the bot `sunfish-engine`. What makes it an unusual verification target is not its size but its *documentation*: the docstring of its central routine, `Searcher.bound`, is a complete two-sided fail-soft contract, stated a decade before anyone tried to prove it. The comments scattered through the search — “`lower <= s(pos) <= upper`”, “the opponent will for sure just stand

¹<https://github.com/thomasahle/sunfish>

pat”, “sunfish may report ‘mate’, but then after more search realize it’s not a mate after all” — turn out to be, respectively: a table invariant, a dischargeable lemma, and the exact failure mode of a hypothesis that a machine-checked counterexample shows to be necessary. The docstrings were latent formal specs.

This paper reports a campaign, conducted in a single day on the live repository, that ran two loops in parallel:

- **The proof loop.** Every search feature was transcribed into a small, dependency-free Lean 4 model (core Lean plus the *omega* tactic; no Mathlib; a full build takes seconds) and either proven sorry-free, refuted by an explicit machine-checked counterexample, or reduced to a *named hypothesis* whose statement — not whose proof — is the deliverable.
- **The price loop.** Every engine change motivated by the proofs (and every change the proofs advised against) was priced by an ELO tournament between frozen engine builds, under the methodology of Section 9, on the same engine that plays real games on Lichess.

The two loops answer different questions and neither substitutes for the other. A proof tells you *what* a pruning trick costs in soundness — which invariant it weakens, at which positions, under which window. A tournament tells you what that weakening costs (or earns) *in ELO*. Several of our measured numbers (Table 3) show soundness restorations that are strictly ELO-negative, and unsound-in-the-point-sense optimizations that are strongly ELO-positive; the formal work is what lets us merge the latter with our eyes open, with a two-line containment for the specific inconsistency the proof predicts — and, once the interval reading of that containment was recognized as guaranteeing nothing, to retire the optimization deliberately at a measured price (Section 7.4) — and finally, when a last measurement campaign showed that the honest form of the optimization is worth exactly zero, to delete the mechanism outright, formal apparatus and all (Section 7.5).

Contributions. (1) A sorry-free Lean 4 proof that sunfish’s fail-soft null-window search satisfies its own docstring (Section 4) — to our knowledge the first Lean 4 formalization of null-window/MTD-bi search. (2) A faithful model of the engine’s king-capture convention and stalemate correction, with a proof under four individually necessary hypotheses and a machine-checked counterexample showing the key hypothesis cannot be dropped (Section 5). (3) A same-day proof of a maintainer-conjectured invariant: the stored killer move at a king-capturable position is itself a king capture (Section 6). (4) A soundness taxonomy for selective search (Section 7), with futility pruning and Late Move Reductions as the canonical citizens of its two nontrivial classes. (5) The ledger-of-bets discipline (Section 8) and the empirical methodology with honest numbers, including artifacts we initially believed (Section 9).

Reading guide. Section 2 defines, once and explicitly, the model in which every theorem of this paper is stated — what is a Lean definition, what is a named hypothesis, what is not modeled at all — so that each later “Theorem” can be read with its exact scope in mind. Section 3 then inventories every machine-checked result: Table 1 lists the proved theorems with their Lean names and files, and Table 2 lists the machine-checked *counterexamples*, which are deliverables of equal rank. The remaining sections tell the story of the main results in the order they were found.

2 The Model: What the Theorems Quantify Over

Every result in this paper is a theorem about a small Lean 4 development (`formal/` in the `sunfish` repository: six files, no dependencies beyond core Lean and the `omega` tactic), not about the Python text of `sunfish.py`. This section fixes the model once, so that each later theorem can be read with its exact scope in mind; how the model-code gap is managed is discussed in Section 11, and Figure 1 shows the correspondence at a glance.

2.1 Abstract games, king-capture legality, and the true value

The base structure is chess-free:

Definition 2.1 (Game). A *game* G consists of a type of positions Pos , a successor function $\text{moves} : \text{Pos} \rightarrow \text{List Pos}$, and a static evaluation $\text{eval} : \text{Pos} \rightarrow \mathbb{Z}$. The empty move list means *the side to move has lost*. (`Game`, `GameTree.lean`)

Sunfish is a *king-capture* engine: it generates pseudo-legal moves, and legality is enforced one ply late by making king capture decisive. The model adopts this convention wholesale: a position with $\text{eval}(p) \leq -M_L$ has already lost its king, and every search in the development normalizes such positions to the exact sentinel $-M_U$. “Check”, “mate” and “stalemate” are therefore not primitives of the model; they are emergent, exactly as in the engine (Section 5). Features that need more structure extend `Game` conservatively: a *NullGame* adds a pass move $\text{pass} : \text{Pos} \rightarrow \text{Pos}$ (the model of `pos.rotate(nullmove=True)`); a *ValGame* adds per-move values val together with sunfish’s incremental-score identity $\text{eval}(m) = -(\text{eval}(p) + \text{val}(p, m))$ as a structural field (Section 7); killer- and transposition-table models additionally assume decidable equality of positions, for table keying.

The “true score” s^* of the docstring is *defined* to be depth-limited negamax: $\text{negamax}_d(p) = \text{eval}(p)$ at $d = 0$, otherwise the maximum of $-\text{negamax}_{d-1}(m)$ over $m \in \text{moves}(p)$, folded from $\text{LOSS} = -M_U$ (`negamax`, `GameTree.lean`). Variants of the value function — `negamaxDraw` (draw-aware, Section 5) and `nullValue` (null-move- and repetition-augmented, Section 8.1) — are likewise definitions, not axioms. (Two further variants have been *deleted* from the development along with the mechanisms they specified: the $(V^{\text{hi}}, V^{\text{lo}})$ pair once defined for re-search LMR — Section 7 tells that story and states the doctrine behind the deletion — and `negamaxDet`, the per-move-reduced-depth value of the deterministic-LMR era, deleted when all reductions were removed at commit 7fdd741, Section 7.5.)

2.2 The searches are Lean functions, one per feature

`Searcher.bound` is modeled as a total recursive Lean function mirroring the Python control flow, and each search feature is modeled in its own file as a variant of that function: the bare fail-soft skeleton `bound` (`Bound.lean`); table threading (`boundTT`), futility (`boundFut`) and mate-score softening in `Tricks.lean`; the stalemate correction (`boundStale`, `Stalemate.lean`); the killer table as state (`boundKill`, `Killer.lean`); and the `can_null`-aware search with null move, repetition check, keyed table and IID (`boundNullTT`, `CanNull.lean`). Two further models existed while the mechanisms they specified shipped — deterministic LMR (`boundLmrDet`, `LmrDet.lean`) and the historical re-search LMR (`boundLmr`, `Lmr.lean`) — and were deleted with those mechanisms at commit 7fdd741 (Section 7.5); git is the archive. Studying features in separate models is itself a modeling decision, and where features could interact the development proved they do not: while the reductions existed, futility (depth ≤ 1) and LMR (depth

≥ 3) never co-occurred (`futilityLmrDisjoint`) and reductions never touched king captures (`RedRespectsCaptures`) — both theorems proven in `Lmr.lean` and now in git history with it — which is what let the killer and stalemate arguments survive under LMR while it shipped.

2.3 Nothing is axiomatized; conditions are named hypotheses

The development contains no `axiom` declarations and no `sorry`. Everything above is a Lean *definition*; every theorem is proven from those definitions, and `#print axioms` on the main theorems reports only `propext` and `Quot.sound` (the ambient axioms of Lean’s kernel). Where a claim about the engine is only *conditionally* true, the condition is not assumed globally: it is a *named hypothesis carried in the theorem statement* — e.g. *Bounded* (evaluations lie in $[-M_U, M_U]$), *CheckProbeOK*, *MateValuesAreKingCaptures*, *NullBetOK*, *FutilityMateOK* — and Section 8 maintains the full ledger. Machine-checked counterexamples are concrete finite games (three to seven positions) whose search values are computed by Lean’s kernel via `decide`; they are theorems of the form “this universally quantified statement is false”, not informal counterarguments.

2.4 What is *not* modeled

Theorems quantify over *all* games of Definition 2.1; sunfish’s chess is one intended instance, but board representation and move generation are entirely abstract. Explicitly outside the model, each with the reason the omission is harmless (audited against master at commit `9b1a7b4` and re-audited after the removal of all reductions at `7fdd741`, `sunfish.py` lines 279–474):

- **Move generation and the board.** `moves` is an opaque function; no chess rules are formalized. Consequently the theorems cannot certify that `gen_moves` implements chess — only how the search treats whatever moves it is given.
- **The quiescence interior.** Depth 0 returns eval directly; QS’s stand-pat-plus-captures recursion is collapsed into the evaluation. Its one load-bearing property, the stand-pat/score identity, is retained as the *ValGame* structure and is exactly what the futility proof consumes (Theorem 7.1).
- **Timing and aborts.** The deadline exception `Stop` raises at node *entry*, before any store, so an abort can leave a search unfinished but never a table entry unjustified — table invariants cannot be corrupted, and time is otherwise irrelevant to the specs.
- **Table eviction.** `TABLE.SIZE` eviction only forgets entries, which trivially preserves every table invariant stated here.
- **Move ordering**, except its load-bearing consequence: king captures (value $\geq M_L$) sort strictly first (`orderedMoves`, `Killer.lean`).
- **The iterative-deepening driver’s scheduling** (when to stop deepening, time allocation). The driver’s *bracket logic* is modeled — it is exactly what Theorem 4.2 serves (Section 4.2).

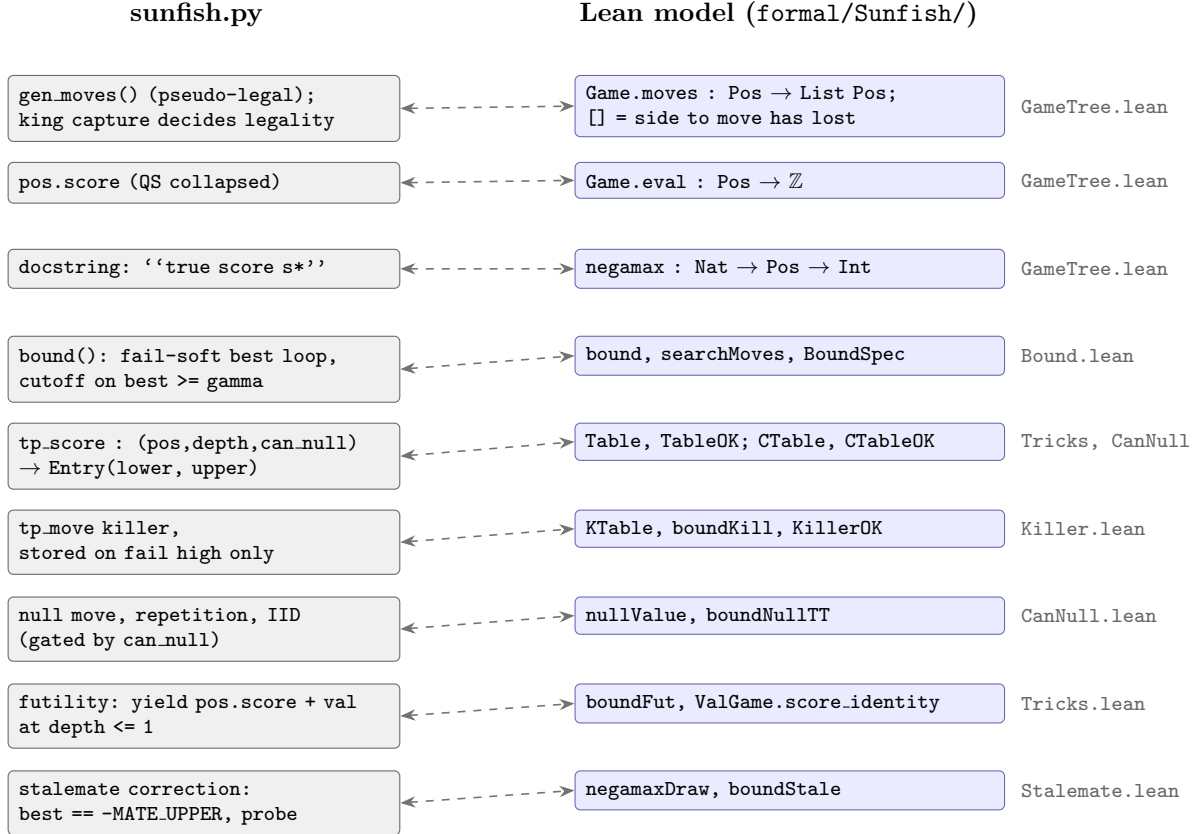


Figure 1: The model–code correspondence, feature by feature. Left: the mechanism in `sunfish.py` (audited at commit 9b1a7b4, re-audited at 7fdd741 after the removal of all reductions); right: the Lean definition that transcribes it, with its file. The full line-by-line map is the correspondence table in `formal/README.md`. Everything on the right is a *definition*; the theorems of Table 1 are proven about these definitions with no axioms beyond Lean’s kernel.

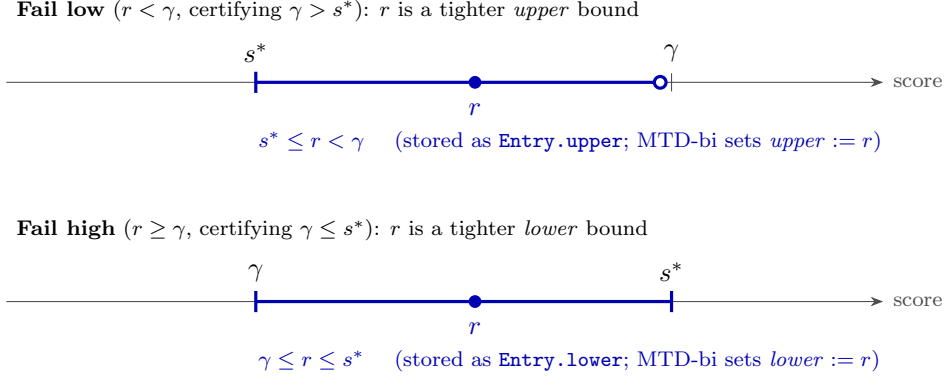


Figure 2: BoundSpec (Definition 4.1) as a picture: wherever the report r falls relative to the probe window γ , it is trapped on the correct side of the true value $s^* = \text{negamax}_d(p)$. Fail-soft means r carries information beyond the boolean fail-high/fail-low: it is itself a valid bound, which is what the transposition table stores and what MTD-bi’s bracket consumes (Figure 3).

3 What Is Proved: The Theorem Inventory

Tables 1 and 2 inventory every machine-checked result cited in this paper, by Lean name and file. The separation matters: Table 1 lists *proved theorems* — positive soundness statements, unconditional unless a named hypothesis is listed — while Table 2 lists *machine-checked counterexamples*: kernel-checked refutations of statements one might have hoped were theorems. Both build sorry-free with `lake build; grep -rn sorry over formal/Sunfish/` finds nothing outside prose.

4 The Core Contract

4.1 The specification, transcribed verbatim

The docstring of `Searcher.bound` (sunfish.py, lines 286–290) reads:

```
Let s* be the "true" score of the sub-tree we are searching.
The method returns r, where
if gamma > s* then s* <= r < gamma (A better upper bound)
if gamma <= s* then gamma <= r <= s* (A better lower bound)
```

In the model of Section 2, the “true score” s^* is the depth-limited negamax value $\text{negamax}_d(p)$, and the docstring transcribes to a two-sided fail-soft condition on the report r , illustrated in Figure 2. Here $M_U = 69290$ and $M_L = 50710$ are sunfish’s `MATE_UPPER` and `MATE_LOWER`.

Definition 4.1 (BoundSpec, the docstring). For a game G , depth d , position p , window γ and report r ,

$$\text{BoundSpec}(G, d, p, \gamma, r) \equiv (\gamma \leq r \rightarrow r \leq \text{negamax}_d(p)) \wedge (r < \gamma \rightarrow \text{negamax}_d(p) \leq r).$$

(BoundSpec, Bound.lean)

Lean theorem	Hypotheses	Informal statement
<code>Bound.lean</code> — the core contract (Section 4)	—	
<code>bound_spec</code> (Thm 4.2)	—	The fail-soft null-window search satisfies its docstring against negamax, at every depth, position and window.
<code>searchMoves_spec</code>	—	Loop invariant: the running <i>best</i> is a fail-soft-correct report of the running true maximum.
<code>Tricks.lean</code> — transposition table, futility, mate tricks (Sections 7, 8)		
<code>boundTT_spec</code>	Bounded	Table threading preserves TableOK (every stored entry brackets $\text{negamax}_d(p)$) and satisfies BoundSpec.
<code>futilityOK_discharged</code> (Thm 7.1)	score identity	The futility yield <i>equals</i> the depth-0 child search it replaces.
<code>boundFut_spec</code> (Thm 7.2)	FutilityMateOK, in-band γ	The futility-pruned search satisfies BoundSpec against plain negamax — a point spec.
<code>soften_null_window</code>	$\gamma > M_L + 1$	Mate-distance softening never changes a fail-high decision.
<code>mateEntry_deep_service</code>	MateDepthMonotone	A mate-band lower bound may be served at any <i>deeper</i> depth.
<code>mateDepthStable_of_kingGoneStable</code>	KingGoneStable, MVAKC	Depth-stability of mate-band membership follows from permanence of king capture.
<code>Stalemate.lean</code> — king-capture model and stalemate correction (Section 5)		
<code>boundStale_spec</code> (Thm 5.4)	Bounded, Check-ProbeOK, in-band γ , MVAKC	The stalemate-corrected search satisfies BoundSpec against the draw-aware value negamaxDraw.
<code>boundStale_of_capture</code>	—	King-capturable positions ($d \geq 1$) return exactly the M_U sentinel.
<code>negamaxDraw_bounded</code>	Bounded	Draw-aware values stay in the $[-M_U, M_U]$ band.
<code>Killer.lean</code> — the killer-table invariant (Section 6)		
<code>boundKill_spec</code> (Thm 6.2)	in-band γ , captures sort first	One call preserves KillerOK and returns exactly M_U at every king-capturable node.
<code>killerEmpty_OK</code>	—	The invariant holds from the empty table along any execution history.
<code>killer_probe_sound</code>	—	The depth-0 probe decides king-capturability (both directions) at the depth the engine runs it.
<i>Deleted with their mechanisms</i> (commit 7fdd741; Sections 7 and 7.5): <code>LmrDet.lean</code> — deterministic LMR — proved <code>boundLmrDet_spec</code> (a point BoundSpec against the single value function <code>negamaxDet</code>), <code>boundLmrDet_no_crossing</code> , and the <code>clamp_noop_high/low</code> no-op theorems; <code>Lmr.lean</code> — re-search LMR (58883ea..7f9f164) — proved <code>bound_no_crossing</code> (Thm 7.3) and the crossing counterexample <code>lmr_tt_crossing</code> (Cex 7.4). Earlier still, <code>boundLmr_spec</code> , <code>lmrVal_sandwich</code> , and <code>IntervalTableOK</code> with its preservation theorems stated an “interval spec” for re-search LMR and were <i>deleted as vacuous</i> (Section 7): no bound on the $V^{\text{hi}} - V^{\text{lo}}$ gap is provable, so they guaranteed nothing; the clamp model (<code>TableClamp.lean</code>) went with them once the clamp was proven a no-op under point specs. Git is the archive; the development carries no historical artifacts.		
<code>CanNull.lean</code> — null move, repetition, keyed table, IID (Section 8.1)		
<code>boundNullTT_spec</code>	Bounded	Layer 1, no zugzwang hypothesis: the augmented search brackets its own value <code>nullValue</code> (point spec) and the (pos, depth, can_null)-keyed table stays consistent.
<code>ctableOK_empty</code>	—	The empty keyed table satisfies the invariant for <i>any</i> history (why <code>tp_score</code> is cleared on history change).
<code>nullValue_plain</code>	NullBetOK, empty history	Layer 2: <code>nullValue</code> collapses to the null-free value; composed with layer 1, this recovers the docstring.

Table 1: **Proved theorems** (sorry-free; axioms `propext`, `Quot.sound` only). “—” means unconditional over all games of Definition 2.1; listed hypotheses are named premises carried in the Lean statement (Section 8). MVAKC = MateValuesAreKingCaptures; in-band means $-M_U < \gamma \leq M_U$. Supporting lemmas are omitted; `formal/README.md` has the complete list.

Lean counterexample	Refutes	Witness
<code>lmr_tt_crossing</code> (<code>Lmr.lean</code> , deleted with the mechanism at 7fdd741 — git history; Cex 7.4, Fig. 5)	Point-consistent <code>tp.score</code> entries under re-search LMR.	3-position game: the same (pos, depth) yields a fail-high report +10 strictly above a fail-low report −50.
<code>boundStale_not_unconditional</code> (<code>Stalemate.lean</code> ; Cex 5.5, Fig. 4)	Soundness of the stalemate correction <i>without</i> <code>MateValuesAreKingCaptures</code> (hypotheses (i)–(iii) alone do not suffice).	7-position game, $\gamma = -5$: the corrected search returns an “upper bound” −20 while the draw-aware value is 0.
<code>negamaxDraw_depth_inconsistent</code> (<code>Stalemate.lean</code> ; Thm 5.6)	Existence of a depth-independent game value agreeing with the depth-gated <code>negamaxDraw</code> .	A stalemate position worth <i>LOSS</i> at remaining depth 2 and 0 at depth 3.
<code>cex_violates_hypothesis</code> (<code>Stalemate.lean</code>)	(Companion fact:) that the 7-position game satisfies Hypothesis 5.3.	<code>negamaxDraw₂(m_0) = M_U</code> with no king capture behind it — a fabricated mate.
<code>extended_value_not_key_independent</code> (<code>Tricks.lean</code>)	<code>ExtKeyIndependent</code> : that the extended-search value is a function of (pos, depth) alone.	2-state game with a history-dependent extension: same key, two values; hence such a TT key must include the history component.

Table 2: **Machine-checked counterexamples.** Each is a concrete finite game whose search values Lean’s kernel computes by `decide`; each is a theorem of the form “the quantified claim is false”. They carry the same evidential weight as Table 1 and did the most to correct the authors’ beliefs (Section 11).

The search itself, stripped to its logical skeleton, is the fail-soft loop with the early cutoff on $best \geq \gamma$, over children searched with the flipped null window $1 - \gamma$ one ply shallower (`bound`, `searchMoves`, `Bound.lean`). No table, no null move, no quiescence, no killer: those return in Sections 5–8.

Theorem 4.2 (The docstring is true). *For every game G , depth d , position p and window $\gamma \in \mathbb{Z}$, $\text{BoundSpec}(G, d, p, \gamma, \text{bound}_d(p, \gamma))$ holds. (`bound_spec`, `Bound.lean`)*

Proof sketch. Induction on depth. The loop is handled by a single invariant lemma (`searchMoves_spec`, `Bound.lean`): *the running best is itself a fail-soft-correct report of the running true maximum.* On a cutoff, $best \geq \gamma$ came either from the current child (whose report is $\leq$ its true value by the induction hypothesis) or from the previous $best$ (bounded by the invariant); either way the fold over the unsearched tail can only grow. Without a cutoff, both the child and the previous $best$ failed low, so both upper-bound their true counterparts and the invariant is re-established. The null-window step is the integer fact $b < 1 - \gamma \iff \gamma \leq -b$: a child failing low against $1 - \gamma$ fails high for the parent, and vice versa. The proof is sorry-free; `#print axioms` reports only `propext` and `Quot.sound`. $\square$

4.2 Why this is the load-bearing spec: the MTD-bi driver

Sunfish’s driver `Searcher.search` is MTD-bi (a bisecting variant of $\text{MTD}(f)$ [8]): at each depth it maintains a bracket $lower \leq score \leq upper$, initialized to $[-M_L, M_L]$, and repeatedly probes `bound` at $\gamma = \lfloor (lower + upper + 1)/2 \rfloor$, stopping when the bracket is narrower than `EVAL_ROUGHNESS` (= 15 centipawns) — Figure 3. Theorem 4.2 is exactly the property the binary search needs: *every* probe, whatever its γ , moves one end of the bracket — a fail high raises *lower* to r , a fail low lowers *upper* to r , and the two can never cross (Theorem 7.3 below makes the no-crossing claim formal, and Section 7 shows how the since-removed re-search Late Move

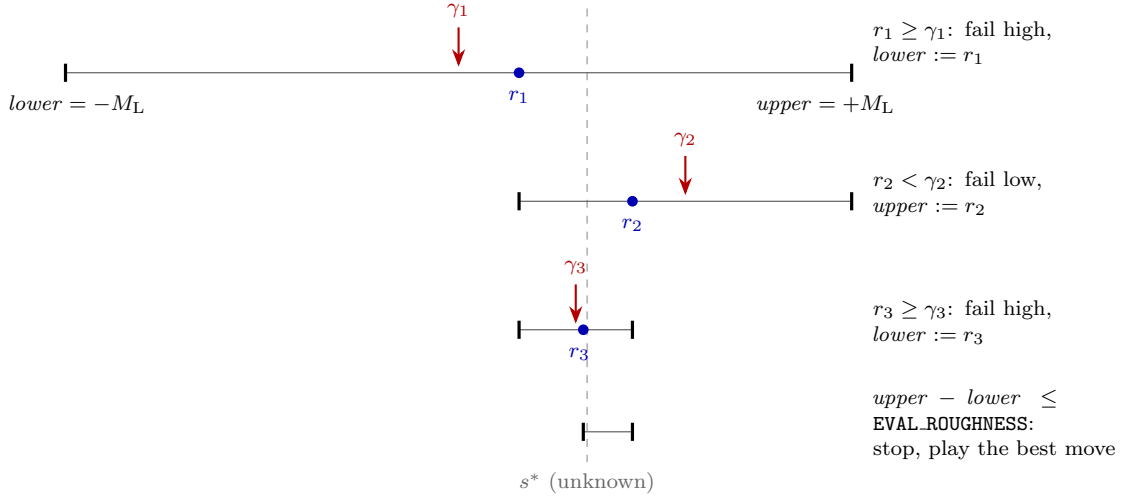


Figure 3: MTD-bi as a binary search over the probe window γ . Each row is one call $r_i = \text{bound}(\text{pos}, \gamma_i, d)$ with $\gamma_i = \lfloor (lower + upper + 1)/2 \rfloor$. Theorem 4.2 guarantees each report lands on the correct side of the (unknown) true value s^* , so every probe legitimately moves one end of the bracket, and fail-soft overshoot (e.g. $r_1 > \gamma_1$) shrinks it faster than plain bisection. The invariant $lower \leq s^* \leq upper$ is Theorem 7.3; Cex. 7.4 showed what the since-removed re-search LMR did to it.

Reductions broke precisely it). Fail-softness is what makes bisection converge fast in practice: the report r typically overshoots γ , so the bracket shrinks by more than half. Two details of the driver are load-bearing for later sections: the probed window range stays inside $(-M_U, M_U]$ — an interval closed under the null-window flip $\gamma \mapsto 1 - \gamma$ — which Theorems 5.4 and 6.2 require; and the bracket invariant is exactly what the LMR crossing counterexample (Cex. 7.4) broke — and, because the spread of the crossing reports admits no provable bound, there was no slack (not EVAL.ROUGHNESS, not re-probing) that could honestly be said to absorb it (Section 7) — the fact that eventually killed the mechanism (Section 7.5).

5 The King-Capture Model and the Stalemate Correction

Sunfish generates *pseudo-legal* moves and scores an actual king capture as an immediate win: a position whose static score is $\leq -M_L$ has already lost its king and is normalized to exactly $-M_U$. “No legal moves” (mate *or* stalemate) therefore surfaces in the search as $best = -M_U$ after the loop, and the engine post-processes it (at depth > 2) with an in-check probe on the null-moved position: $best := -M_L$ if in check, else 0.

Definition 5.1 (Draw-aware value). $\text{negamaxDraw}_d(p)$ is negamax over the king-capture-normalized game with the depth-gated stalemate correction applied to the fold: if $d > 2$ and the fold equals *LOSS*, the value becomes $-M_L$ if p is in check and 0 otherwise. (`negamaxDraw`, `drawFix`, `Stalemate.lean`)

Definition 5.2 (CheckProbeOK). A probe function agrees pointwise with the one-ply king-capture reading of check: after passing the turn, the opponent has a move reaching a position with $\text{eval} \leq -M_L$. For the real engine this is the assumption that the depth-0 QS probe

`bound(flipped, MATE_UPPER, 0) == MATE_UPPER` decides `check` — `true` because quiescence never prunes king captures. (`CheckProbeOK`, `inCheckB`, `Stalemate.lean`)

Named Hypothesis 5.3 (`MateValuesAreKingCaptures`). Whenever $\text{negamaxDraw}_d(p) = M_U$ with $d \geq 1$, some move of p actually reaches a position with $\text{eval} \leq -M_L$: mate-band sentinel values always come from a real king capture, never from a shallow-horizon artifact. (`MateValuesAreKingCaptures`, `Stalemate.lean`)

Theorem 5.4 (The corrected search satisfies the docstring). *Let G be a game with a pass move, and suppose: (i) all evaluations lie in $[-M_U, M_U]$ (`Bounded`); (ii) the probe satisfies `CheckProbeOK`; (iii) the window is in band: $-M_U < \gamma \leq M_U$ (an interval closed under the null-window flip $\gamma \mapsto 1 - \gamma$); and (iv) `MateValuesAreKingCaptures`. Then the stalemate-corrected search `boundStale` satisfies `BoundSpec` against `negamaxDraw` at every depth and position.* (`boundStale_spec`, `Stalemate.lean`)

Proof sketch. The same loop invariant as Theorem 4.2, plus a pointwise lemma (`staleFix_spec_core`, `Stalemate.lean`) showing the correction maps a fail-soft-correct pair (search *best*, *true* fold) to a fail-soft-correct pair. The crux is the masking step: if the *true* fold is *LOSS*, then (by the band, the sentinel invariant, and Hypothesis 5.3) every child is an exact king-capture mate, every child report is exactly *LOSS*, and hence the search’s *best* is *LOSS* too — so both sides apply the *same* correction, using the correct probe. In-band windows are needed so that a fail-low at *LOSS* certifies the loop ran to completion: a $\gamma \leq -M_U$ would allow an early cutoff at *LOSS* with unsearched moves remaining. Sorry-free. $\square$

Each hypothesis is individually necessary; the interesting one is (iv):

Counterexample 5.5 (The correction is not unconditionally sound). There is a seven-position game with bounded honest evaluations, a perfectly correct probe, and an in-band window $\gamma = -5$, on which the faithful depth-gated correction returns a fail-low “upper bound” of -20 while the draw-aware value is 0 . The mechanism: two positions are terminal stalemates; seen at remaining depth ≤ 2 (gate off) they score *LOSS*, so their parents at depth 2 score a fabricated M_U — mate scores with no king capture anywhere behind them — and the root inherits a wrong bound. This is sunfish’s own code comment (“sunfish may report ‘mate’, but then after more search realize it’s not a mate after all”) turned into a machine-checked refutation. (`boundStale_not_unconditional`, `Cex`, `cex_violates_hypothesis`, `Stalemate.lean`)

Theorem 5.6 (Depth inconsistency is intrinsic). *The depth > 2 gate makes the draw-aware value itself depth-dependent: in the counterexample game there is a stalemate position g with $\text{negamaxDraw}_2(g) = \text{LOSS}$ and $\text{negamaxDraw}_3(g) = 0$. No depth-independent game value can agree with `negamaxDraw` at every depth.* (`negamaxDraw_depth_inconsistent`, `Stalemate.lean`)

Remark 5.7 (Why mate is $-M_L$, not $-M_U$). $-M_U$ is the reserved sentinel meaning “the king is *actually* capturable/gone”. Scoring checkmate-by-correction as $-M_U$ would re-inject the sentinel at positions whose king is not capturable and destroy the parent-level detection one ply up; it would also order “mated next move” above “king already lost”. The engine’s choice of $-M_L$ at line 412 is what keeps the sentinel trustworthy.

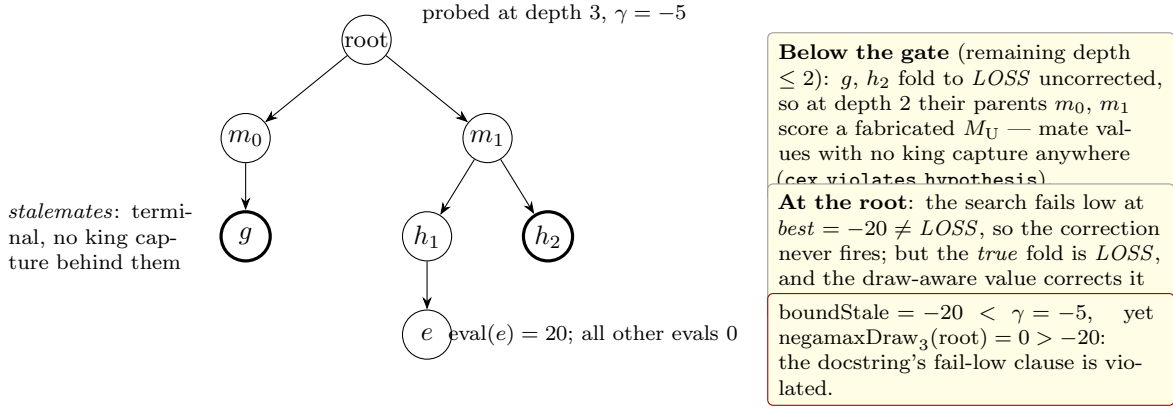


Figure 4: The seven-position counterexample **Cex** (Cex. 5.5). Thick circles are terminal stalemates; the depth > 2 gate means they are scored *LOSS* when first seen at shallow remaining depth, their parents inherit fabricated mate scores, and the root reports an upper bound the draw-aware value exceeds. Bounded evaluations, a perfect check probe and an in-band window all hold; only Hypothesis 5.3 fails — so the hypothesis is not optional.

6 The Killer Invariant: Conjectured at Noon, Proved by Evening

While modeling the stalemate correction we recorded a potential gap: the killer move (the stored `tp_move` entry) is yielded *before* the sorted moves, so a non-capture killer failing high could let `bound` return a value below M_U at a king-capturable position — breaking the sentinel requirement that Theorem 5.4 leans on. The maintainer conjectured this cannot happen in sunfish. The conjecture was proved the same day.

Definition 6.1 (KillerOK, the table invariant). A killer table t (a partial map from positions to moves) satisfies KillerOK if every stored entry is a legal move of its key position, and at a king-capturable position the stored entry — if any — is *itself a king capture*. (KillerOK, `Killer.lean`)

Theorem 6.2 (KillerIsKingCapture). Thread `tp_move` through the search as state (*position-keyed, store-on-fail-high-only*), with the move loop running over value-ordered moves (*king captures, value $\geq M_L$, strictly first*). Then for every depth d , position p , in-band window $-M_U < \gamma \leq M_U$, and table t satisfying KillerOK:

1. the call `boundKilld(p,  $\gamma$ , t)` returns a table satisfying KillerOK; and
2. if $d \geq 1$, p 's own king is not already gone, and p is king-capturable, the returned value is exactly M_U — on the killer path and on the loop path alike.

(`boundKill_spec`, `Killer.lean`)

Proof sketch. Induction on depth, cases on the table lookup. *Killer-cutoff case*: the stored move is the killer itself; by the invariant it is a king capture whenever p is capturable, the king-capture normalization scores it at the full sentinel, and an in-band γ makes the sentinel fail high — so the cutoff both stores a king capture and reports exactly M_U . *Killer-fail-low case*: a king-capture killer would have reported $M_U \geq \gamma$, so p is not capturable and plain loop preservation applies. *No-killer case*: at a capturable position the capture heads the ordered list, cuts off immediately at the

sentinel, and nothing else can be stored first (`killLoop_capture_head`, `Killer.lean`). Applied along any execution history starting from the empty table (`killerEmpty_OK`, `Killer.lean`), the invariant holds forever. Sorry-free; axioms `propext`, `Quot.sound` only. $\square$

Corollary 6.3 (Exact sentinel; fail-low certifies non-capturability). *At any king-capturable node ($\text{depth} \geq 1$, in-band window) the search reports exactly M_U ; contrapositively, any in-band report $< M_U$ — in particular any fail low — certifies that no king capture was available. This is what makes the depth-0 stalemate probe of Section 5 meaningful.*

Three conditions are load-bearing, and each is an edit a well-meaning PR could make:

- **Position-exact keying.** Every store into `tp_move[p]` happens inside a call at p itself; a ply-shared killer table (a common engine idiom) would let a quiet killer harvested at one position cut off at a different, king-capturable one, breaking the induction.
- **Captures sort first.** King captures have value $\geq M_L$, strictly above every quiet move.
- **In-band windows.** An out-of-band $\gamma > M_U$ could let a quiet move cut off (and be stored) above the capture’s sentinel.

The residual exception is the null-move yield: it stores nothing (so `KillerOK` survives) but can end the loop below the sentinel; sunfish guards it only by `|pos.score| < 500`, a guard its own `FIXME` concedes is heuristic. The condition under which the guard closes the hole is the named hypothesis (`NullGuardBlocksAtCaptures`, `Killer.lean`).

Empirical companion. Before the proof existed, the conjecture was checked empirically on the production engine: **0 violations in 694,533 killer cutoffs over 1,270 real-game positions**. The proof upgraded “empirically absent” to “impossible given in-band windows and value ordering” — and, usefully, told us exactly which future edits would void the guarantee.

7 A Soundness Taxonomy for Selective Search

The campaign’s conceptual yield is a classification of how a selective search may consult its window γ . Sunfish’s maintainer stated it as a principle, which we quote verbatim:

“ γ may shape termination, and may trigger shortcuts whose value provably one-side-bounds the same function; it must never select between incomparable evaluations.”

The three classes, each with its canonical citizen:

7.1 Class (a): γ -shaped termination — always sound

The beta cutoff itself: γ decides only *when the loop stops*, and fail-soft reporting keeps every report a bound on the same negamax. This is Theorem 4.2; nothing further is needed. Null-window search, iterative deepening, and aspiration re-probing all live here.

7.2 Class (b): γ -triggered shortcuts that one-side-bound the same function — sound

Futility pruning is the canonical citizen. At depth ≤ 1 , once $\text{pos.score} + \text{val}(m) < \gamma$, sunfish answers with the *static estimate* $\text{pos.score} + \text{val}(m)$ instead of searching the child. The estimate is below γ , so it is only ever a fail-low report, and BoundSpec demands of a fail-low report exactly one inequality: $-\text{negamax}_d(m) \leq \text{pos.score} + \text{val}(m)$.

We initially stated that inequality as a hypothesis (*FutilityOK*). It was then *discharged*: sunfish builds child positions incrementally, and the score identity

$$\text{eval}(m) = -(\text{eval}(p) + \text{val}(p, m)) \quad (m \in \text{moves}(p))$$

is literal in `Position.move`. With the identity as a structural property of the model (`ValGame`, `Tricks.lean`), the futility test $\text{pos.score} + \text{val} < \gamma$ is integer-equivalent to “the child’s stand-pat meets its window $1 - \gamma$ ”, at depth 0 the stand-pat is yielded first and fails high immediately, and fail-soft returns exactly the child’s score:

Theorem 7.1 (*FutilityOK discharged — as an equality*). *In any game with the score identity, for every position p and move $m \in \text{moves}(p)$,*

$$-\text{negamax}_0(m) = \text{eval}(p) + \text{val}(p, m).$$

The futility yield is not an estimate: it equals the depth-0 child search it replaces. (`futilityOK_discharged`, `Tricks.lean`)

Theorem 7.2 (*Futility-pruned search satisfies BoundSpec*). *For any game with the score identity, under in-band windows and the remaining hypothesis FutilityMateOK (a move valued as a king capture, $\text{val} \geq M_L$, really wins: the fail-high `else MATE_UPPER` bypass), the futility-pruned search satisfies BoundSpec against plain negamax — a point spec on a single value function. (`boundFut_spec`, `Tricks.lean`)*

Proof sketch. At depth 1 (the futility zone) each per-move yield is analyzed by cases: a futile quiet move yields exactly $-\text{eval}(m)$ by Theorem 7.1; a futile king capture yields the exact sentinel, a fail high justified by *FutilityMateOK* and the in-band window; a non-futile move is an exact depth-0 child. A member-restricted variant of the loop lemma (`searchMoves_spec_mem`, `Tricks.lean`) (the score identity only speaks about legal moves) closes the loop; deeper depths are Theorem 4.2’s argument verbatim. Sorry-free. $\square$

Fine print made explicit by the model: the *quantified* *FutilityOK* (over all depths) is not dischargeable — plain negamax at $d \geq 1$ has no stand-pat option — but the search only ever consumes the $d = 0$ instance, and that instance is the score identity on the nose. The original statement over-required.

7.3 Class (c): γ -selected incomparable evaluations — unsound in the point sense

Re-search Late Move Reductions — the gamma-adaptive variant that shipped between commits 58883ea and 7f9f164, and whose formal record was kept in `Lmr.lean` until mechanism and model were deleted together at 7fdd741 (Section 7.5; git history) — is the canonical citizen. A late quiet move at depth ≥ 3 is probed at depth -2 ; a reduced fail low is yielded as-is, a reduced

fail high is re-searched at full depth and only that result is yielded. Here γ *selects which of two incomparable value functions the yield is a fact about*, and the docstring becomes unprovable as stated: no single value function backs both sides.

Our first reaction was to retreat to an interval-shaped statement. Define the mutually recursive pair $(V^{\text{hi}}, V^{\text{lo}})$ with side alternation:

$$\begin{aligned} V_d^{\text{hi}}(p) &= \max_m -V_{d-1}^{\text{lo}}(m) && \text{(all full depth)} \\ V_d^{\text{lo}}(p) &= \max_m \begin{cases} \min(-V_{d-2}^{\text{hi}}(m), -V_{d-1}^{\text{hi}}(m)) & \text{if } m \text{ reducible} \\ -V_{d-1}^{\text{hi}}(m) & \text{otherwise} \end{cases} \end{aligned}$$

over an abstract reducibility predicate $\text{red}(d, i, m)$ generalizing sunfish’s `depth >= 3 and i_m >= 5 and val < QS`. We proved, machine-checked and sorry-free at the time (as `boundLmr_spec` and `lmrVal_sandwich`), that every fail-high report of the LMR search is $\leq V_d^{\text{hi}}(p)$, every fail-low report is $\geq V_d^{\text{lo}}(p)$, and $V_d^{\text{lo}}(p) \leq \text{negamax}_d(p) \leq V_d^{\text{hi}}(p)$ pointwise — and, for a while, we presented this pair of statements as “the honest replacement” for the docstring: an *interval spec*, with the truth trapped in between.

The verdict must be stated as plainly as the retreat was: **it was never a spec**, and the statements — together with the $(V^{\text{hi}}, V^{\text{lo}})$ definition itself — have been *deleted* from the development. A claim of the form “fail highs bound V^{hi} , fail lows bound V^{lo} , and the truth lies in between” is a guarantee only if the gap $V^{\text{hi}} - V^{\text{lo}}$ is bounded. No bound on search instability is provable: one ply of depth can hide a mate, so the gap is unbounded precisely in the positions that decide games. A guarantee relative to an uncontrollable gap guarantees nothing; the interval reading was only a description of the bug — in the maintainer’s phrase, a “cope spec”. The only contract the repository recognizes is the *point spec*: every stored bound describes one value function determined by the transposition key. What `Lmr.lean` kept, until the mechanism itself was removed and the file deleted with it (Section 7.5), was exactly what supported that doctrine: the re-search definition itself (`boundLmr`) and the machine-checked counterexample below.

Two facts about modeling the since-deleted statements were *surprises* — both were claims the authors believed and the proof assistant refused; we recount them honestly in Section 11.

The practical consequence is a transposition-table pathology:

Theorem 7.3 (No crossing without LMR). *For the unreduced search, a fail-high report can never exceed a fail-low report at the same (pos, depth): both bracket $\text{negamax}_d(p)$, so a `tp_score` entry always satisfies $\text{lower} \leq \text{upper}$. (`bound_no_crossing`, proved in `Lmr.lean` and deleted with the file at `7fdd741`: for the now-shipped full-width search the fact is immediate from Theorem 4.2 — both reports bracket the same $\text{negamax}_d(p)$.)*

Counterexample 7.4 (TT crossing under LMR, machine-checked). There is a three-position game and windows $\gamma = -60, 0$ on which the LMR search at the same (pos, depth) produces a fail-high report of +10 and a fail-low report of -50: a `tp_score` entry with $\text{lower} = 10 > -50 = \text{upper}$. The single root move looks bad shallow but is good deep; probing $\gamma = 0$ takes the reduced branch and fails low at -50, probing $\gamma = -60$ bypasses it, re-searches at full depth and fails high at +10. (`lmr_tt_crossing`, `Lmr.lean`; deleted with the mechanism at `7fdd741` — in git history at `58883ea..7fdd741`.)

MTD-bi’s bracket $\text{lower} \leq \text{score} \leq \text{upper}$ is therefore simply broken under re-search LMR: the gap between the two value functions admits no provable bound, so there is no theorem of

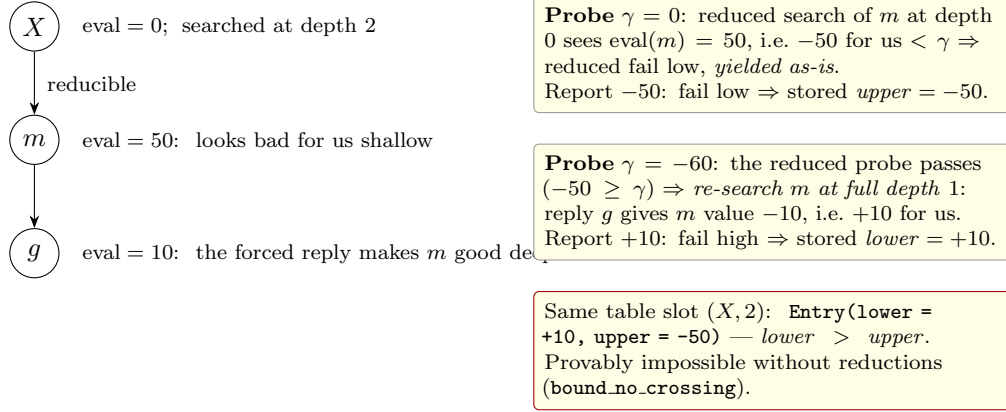


Figure 5: The machine-checked TT-crossing counterexample `lmr.tt.crossing` (Cex. 7.4). The window γ decides whether the yield for move m is a fact about the *reduced* (depth-0) or the *full* (depth-1) evaluation — two incomparable value functions — so two probes of the same (pos,depth) return bounds that cross when stored. Mechanism and Lean artifact are both gone from the working tree (Section 7.5); the counterexample survives in git history — and here, as the explanation of why re-search LMR died.

the form “the crossing stays within what EVAL.ROUGHNESS and re-probing absorb” — in the positions that decide games, it need not. On the production engine the crossing is real but rare: **0.016% of nodes** measured over real games. The two-line store clamp shipped at the time — on store, widen the stale side of the entry instead of asserting a contradiction — measured ELO-neutral at $+12 \pm 28$; but it was containment, not repair: it kept crossing reports from being *stored* (proved at the time as `clamp.no.crossing`) while the search went on consuming them. The `IntervalTableOK` invariant that once dressed the clamp up as preserving a guarantee was deleted along with the interval reading — an invariant stated relative to an unboundable gap certifies nothing — and the clamp model (`TableClamp.lean`) has since been deleted from the development entirely; git is the archive. Killer and stalemate lemmas were unaffected while the mechanism shipped: king captures have $\text{val} \geq M_L > QS = 40$ and were never reducible (`RedRespectsCaptures`, deleted with `Lmr.lean`); the killer is pre-loop and structurally never reduced; reduced fail lows are $< \gamma$ and never reach the `tp.move` store; and the $-M_U$ king-loss sentinel is depth-independent, so stalemate detection still saw exact sentinels from reduced searches.

7.4 First epilogue: deterministic LMR restores the point spec

The story seemed, for a day, to have its resolution. At commit `7f9f164`, master retired the re-search variant and shipped *deterministic* LMR: the reduction depended only on (depth, index, val) — `LMR = int(depth >= 4 and i.m >= 8 and val < 0)` — and each move was searched *once*, at depth $-1 - \text{LMR}$. Because γ no longer selected between evaluations, the search was back in class (b’): it computed exact fail-soft bounds on a *single* value function `negamaxDet` (reducible moves valued one ply shallower, recursively), with a point spec (`boundLmrDet.spec`, `LmrDet.lean` — proven at the time, deleted with the mechanism, Section 7.5); fail-high reports could never exceed fail-low reports at the same key, so transposition entries were contradiction-free by construction (`boundLmrDet.no.crossing`); and the store clamp of Cex. 7.4’s containment became a provable

no-op (`clamp_noop_high/low`, proven at the time) and was removed from the engine — after which the no-op theorems and the whole clamp model (`TableClamp.lean`) were themselves deleted from the development, under the maintainer’s principle that git is the archive and the code carries no historical artifacts. The price was believed measured, not guessed: the gamma-adaptive re-search variant appeared about 16 ELO stronger (-16 ± 38 direct; both large wins over no LMR, or so the ledger then read — Table 3). That trade — 16 ELO for end-to-end point specs on single value functions — was made deliberately. Note that `negamaxDet` $\neq$ `negamax`: deterministic LMR was honest about *which* value function it computed bounds on, rather than unsound about the original one. It was also, as the next subsection shows, mispriced — in both directions at once.

7.5 Second epilogue: the measurement campaign removes the mechanism entirely

The resolution of Section 7.4 did not survive its next measurement campaign (all matches per `docs/TESTING.md`: frozen snapshots, paired openings, per-engine time-loss accounting). Three results, in the order they dismantled the trade:

- **Removing deterministic LMR is a gain, not a price.** No-LMR beat the shipped deterministic LMR by $+69 \pm 40$ and $+29 \pm 33$ (two 300-game matches at 4+0.04) and by $+19 \pm 45$ at 60+1 (200 games) — stronger at every time control tested, with fixed-depth suites corroborating (mate-in-4, Bratko–Kopeck and Win-at-Chess all improve without LMR). Deterministic LMR’s believed -16 ± 38 price against re-search was really ~ -50 : net negative against *no* LMR at all. The methodological lesson went into `docs/TESTING.md`: a decision that hinges on a difference smaller than its error bars needs SPRT or a larger sample.
- **The historical ledger was never wrong.** Replaying the exact A/B that merged re-search LMR at 58883ea ($+49 \pm 34$ then) reproduces today: $+41 \pm 36$.
- **The edge was the bug.** Re-search LMR grafted onto today’s baseline still measures $+20 \pm 36$ over no-LMR — but an *honest* variant with identical cutoffs and sound min-semantics bound propagation (each stored bound a claim about the minimum over the depths actually searched, key-determined by construction) scores exactly 0.00 ± 34 against no-LMR. The entire measured value of re-search LMR lived in its over-claimed bounds: fail highs propagated as facts about a depth they did not search. That is not a trade-off between strength and provability; it is strength borrowed from a bug — the very transposition-table contradiction machine-checked in Cex. 7.4 while the mechanism existed.

Commit 7fdd741 therefore removes Late Move Reductions entirely. The move loop is again a single full-width `for val, move in sorted(...)`, every move searched at depth -1 ; with no reduction anywhere, `bound`’s docstring is provable exactly as written — Theorem 4.2, the point spec of `Bound.lean`, now describes the shipped loop with no caveats — and the formal record needs no LMR machinery: `Lmr.lean`, `LmrDet.lean` and `TableClamp.lean` were deleted with the mechanism, under the same git-is-the-archive principle that had already claimed the interval spec. What survives in the development is what the episode taught, stated once in `formal/README.md`: every stored bound must describe one value function determined by the transposition key; “the truth lies within an unboundable gap” is not a specification.

8 The Ledger of Bets: Named Hypotheses

Every pruning trick that is only conditionally sound is kept out of the proven core and given a *named hypothesis*: the condition, not the induction, is the interesting content. The discipline this buys is a repository rule: **a search-changing PR must identify which lemma it preserves, weakens, or newly depends on** (see `formal/README.md` for the full PR-guideline ledger). Every unproven assumption is named, stated, and where possible measured.

Hypothesis	Statement (informal)	Status
NullOK / NullBetOK	Some real move matches the reduced-depth pass value. <i>Zugzwang</i> is precisely its negation; hence sunfish’s $ \text{score} < 500$ guard.	Named; confined to the layer-2 bridge <code>nullValue.plain</code> (proven under it); layer 1 is unconditional (Sec. 8.1)
TableOK	Every stored $(\text{lower}, \text{upper})$ brackets $\text{negamax}_d(p)$; requires the <i>Bounded</i> side condition, without which the fresh entry $(-M_U, M_U)$ is not a valid bracket and the claim is false.	Proven (<code>boundTT.spec</code> , under <i>Bounded</i>)
FutilityOK	Static estimate dominates the true child value.	Discharged (Thm. 7.1)
FutilityMateOK	A move valued as a king capture really wins everything.	Named (a fact about chess, not arithmetic)
MateValuesAreKingCaptures	Mate-band values come from real king captures (Hyp. 5.3).	Named; necessity machine-checked (Cex. 5.5)
CheckProbeOK	The depth-0 null probe decides check.	Named; true because QS never prunes king captures
MateDepthMonotone	A mate-band lower bound survives one more ply; lifts to arbitrary deeper service (<code>mateEntry.deep.service</code> , <code>Tricks.lean</code>).	Named; lift proven
MateDepthStable	Mate-band membership is depth-independent for $d \geq 1$; what <i>shallower</i> service of sentinel mate entries assumes. Any variant using it should document itself as weakening <code>BoundSpec</code> to mate-band membership.	Named; derivation from <code>KingGoneStable</code> sketched
KingGoneStable	A captured king is permanent, on both sides of the sign alternation.	Named
NullGuardBlocksAtCaptures	The null-move guard fires at no king-capturable position: closes the residual killer exception (Sec. 6).	Named
QuietEvalsInBand	Quiet positions evaluate strictly inside the mate band.	Named
RedRespectsCaptures	Reductions never touch king captures ($\text{val} \geq M_L > QS$).	True by construction in sunfish
ExtKeyIndependent	Extended-search value is a function of $(\text{pos}, \text{depth})$ alone.	Refuted (<code>Tricks.lean</code> ; Table 2)

Two entries deserve a sentence each. `ExtKeyIndependent` is stated as the *false* claim a $(\text{pos}, \text{depth})$ -keyed table silently makes once search extensions depend on history (e.g. a recapture extension keyed on the last-capture square); a two-state counterexample refutes it, and the consequence is that such a TT key must include the history component — sunfish’s `can_null` flag being the same phenomenon in miniature, duly part of its key. And `soften_null_window` (`soften_null_window`, `Tricks.lean`) is a small proven lemma: for windows strictly above the mate band, mate-distance softening does not disturb null-window fail-high decisions.

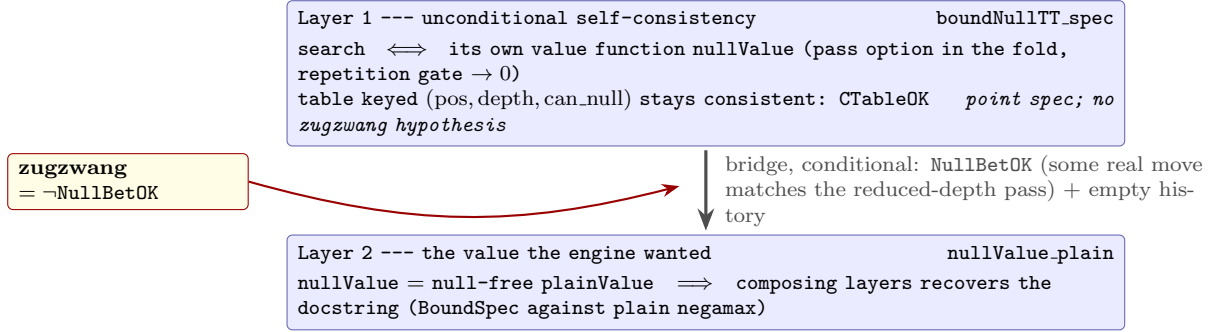


Figure 6: The layered null-move argument (Section 8.1). The unconditional layer proves the search and its keyed table can never disagree with *each other*; the named hypothesis `NullBetOK` is confined to the bridge that identifies the computed value with the null-free one. Zugzwang attacks only the bridge — it can make the value wrong, never the table inconsistent.

8.1 Shrinking a bet: the layered null-move story

The ledger is not static; the null-move entry shows a bet being *shrunk* by restructuring (Figure 6). Sunfish’s `can_null` flag plays four roles at once — null-move gate, repetition gate, transposition-key component, and IID recursion flag — and `CanNull.lean` models all four exactly as master runs them (including two audit surprises recorded there: sunfish permits *consecutive* null moves, and the move generator’s laziness is semantically load-bearing for the table state). The old, monolithic claim “null-move search satisfies the docstring” needed a zugzwang hypothesis in front of everything. The layered version needs it almost nowhere:

- **Layer 1 (unconditional).** Define `nullValue` — the value function of the null-and-repetition-augmented search itself, with the pass option (searched at depth -3 behind the gamma-free guard $|\text{score}| < 500$) as one more option in the fold. Then the search brackets `nullValue` with a *point* spec, and the $(\text{pos}, \text{depth}, \text{can_null})$ -keyed table stays consistent, with no zugzwang hypothesis anywhere (`boundNullTT_spec`, `CTableOK`, `CanNull.lean`). Self-consistency of search plus table is not a bet.
- **Layer 2 (the bet, now smaller).** Relating `nullValue` to the null-free value is where the null-move bet lives: under `NullBetOK` (some real move matches the reduced-depth pass; the required witness also excludes guard-passing stalemates) and an empty history, `nullValue` collapses to the plain value (`nullValue_plain`, `CanNull.lean`), and composing the layers recovers the docstring.

Zugzwang — precisely $\neg \text{NullBetOK}$ — can make the engine compute the wrong *value*, but can never make its table contradict its search. A satisfying corollary of the keyed-table invariant being history-relative: the empty table satisfies it for *any* history (`ctableOK.empty`, `CanNull.lean`), which is exactly why sunfish clears `tp_score` whenever the game history changes.

The framing, in one paragraph: the engine is *allowed* to bet — that is what selective search is — but every bet must have a name, a statement in the ledger, and, where possible, a measured exposure. Restructuring can shrink a bet (this subsection); a counterexample can prove a bet is real (Cex. 5.5); and a discharge can retire one (Theorem 7.1). What is not allowed is an unnamed bet.

9 Empirical Methodology and Honest Numbers

Functional tests tell you the engine is correct; they tell you nothing about whether a change makes it stronger. Every strength-relevant change in this campaign was priced by a tournament between engines frozen with `git archive` (code *and* tools — no shared files), playing color-swapped pairs from a random-order opening book under fastchess [2], typically 300 games at `tc=4+0.04` ($\approx \pm 30$ ELO at 95%), with adjudication, on an otherwise idle machine. The full methodology, each rule with its origin story, is `docs/TESTING.md` in the repository.

9.1 Results

Change	ELO	Disposition
Late Move Reductions (re-search)	$+49 \pm 34$	merged (LOS 99.8%, 300 games); later retired
TT store-clamp (crossing containment)	$+12 \pm 28$	merged (free); later a no-op
Deterministic LMR (first attempt)	-48 ± 33	declined
Deterministic LMR (conservative)	-16 ± 38	declined, then merged (7f9f164)
Sticky-entry consistency restoration	-33	declined
Ordering-only consistency restoration	-34	declined
Null-move soundness completion	-28	declined
Reverse futility	$+45 \rightarrow -1$	artifact (time regime); declined
Mate-score softening	$+19 \rightarrow -22$	artifact (time regime); declined
Removing deterministic LMR	$+69 \pm 40$, $+29 \pm 33$	merged (7fdd741); also $+19 \pm 45$ at 60+1
Re-search LMR, historical A/B replayed	$+41 \pm 36$	the 58883ea +49 reproduces
Re-search LMR vs. no-LMR (today)	$+20 \pm 36$	the unsound variant’s residual edge
Honest (min-semantics) LMR vs. no-LMR	0.00 ± 34	the edge was the bug; not merged

Table 3: Measured prices. Error bars are 95% where quoted; all matches per `docs/TESTING.md` (300 games, `tc=4+0.04`, opening book, adjudication) unless noted. The middle block prices *consistency*: every attempt to restore point-soundness under LMR — making the reduced search deterministic, keeping entries sticky, or restoring ordering only — appeared to cost 16–48 ELO. The proofs say what is being traded; these numbers say what the trade is worth. Dispositions reflect the full history: after the campaign, the maintainer deliberately accepted the cheapest restoration — conservative deterministic LMR at -16 ± 38 — in exchange for end-to-end point specs (Section 7.4); the re-search rows above the line are therefore “merged, later retired”, and the store clamp became a provable no-op and was removed. The bottom block is the final campaign (Section 7.5): deterministic LMR’s believed -16 was really ~ -50 — net negative — and with the honest variant worth exactly zero, all reductions were removed at 7fdd741.

Three narrative points, because the numbers alone mislead:

The price of point-consistency. Re-search LMR admits no point spec (Cex. 7.4), and the interval-shaped statement we first proved in its place guarantees nothing (Section 7). One can buy the point spec back — and each way of doing so was priced: deterministic LMR (-48 ± 33), a conservative reduction condition (-16 ± 38), sticky TT entries (-33), ordering-only (-34). Every restoration appeared ELO-negative. Likewise, completing null-move soundness (searching the pass at a depth for which the spec discharges) was priced at -28 and declined: the engine

keeps the bet, and the ledger keeps its name (`NullBetOK`). The two epilogues are instructive. Knowing both the price *and* what it buys, the maintainer first chose to pay the smallest one — conservative deterministic LMR, believed ~ 16 ELO for point specs end-to-end (Section 7.4). Then the final campaign (Section 7.5) showed there had been nothing to pay for: the believed -16 was really ~ -50 , and the honest form of what the point spec supposedly cost is worth exactly 0.00. Master now ships no reductions at all — the point spec holds for the code as written, at a measured *gain*. Neither decision — first paying, then unwinding — would have been coherent without both loops: the proofs identified exactly what the ELO was buying (unsound bound propagation), and the tournaments certified that buying it honestly earns nothing.

Time-regime artifacts. An earlier survey measured reverse futility at $+45$ and mate-score softening at $+19$. After a fix to time handling (in-search deadlines replacing between-iteration checks), both were re-measured on the fixed baseline: -1 and -22 . The original numbers were artifacts of the old time regime, not chess. Rule: after any time-handling change, re-baseline everything you believed.

The multi-TC lesson. An opening think-time ramp (cap of $0.1s \times$ ply) shipped with no cutoff and silently governed *entire games* at long time controls: at $15+3$ the proper budget is $\sim 25s$ /move but the ramp still bound at move 60, and a user’s game ended in mate with 14 of 15 minutes unused. The bug was invisible at the $4+0.04$ test TC, where the ramp stops binding at ply 2 — it sailed through 300-game matches. It was caught by a user report and fixed the same day; the fix was verified by driving the per-move time curve directly at $15+3$ ($17-24s$ observed mid-game vs. $3-9s$ before). This became a standing rule: time-management changes must be validated at multiple time controls, including a direct per-move time-curve check, because fast-TC matches are structurally blind to long-TC time bugs.

Contamination stories, briefly. A contaminated run (engines sharing a live checkout) once measured -60 ELO at LOS 0.02% for a change later shown to search bit-identical node counts — hence the freeze rule and the old-vs-old control match. Tournament managers reuse engine processes across games, so state surviving `ucinewgame` acts as invisible cross-game learning: a four-way eviction-policy round-robin ranked $+66 / +47 / +15 / -132$ across candidates whose *per-game* search was in some cases provably identical. Behavior-neutrality is checked by depth-limited lockstep (moves *and* node counts), never by clock.

9.2 The frame

Proofs identify the trade; tournaments price it. Neither loop alone would have handled LMR correctly: without the proof we would not have known the crossing existed, that no interval reading rescues it, or that the killer and stalemate arguments survive reduction; without the tournaments, the point spec would first have been bought blind — and we would never have learned that its true price was negative: the honest variant of the reductions is worth exactly zero (Section 7.5), so the engine now ships full-width and provable, stronger than it was with either LMR.

10 Related Work

Nipkow’s invited ITP 2024 paper [7] and the accompanying Isabelle/HOL AFP entry [6] verify fail-hard and fail-soft alpha-beta over linear orders, distributive lattices and de Morgan domains; the fail-soft correctness statement there is the two-sided bound that our BoundSpec specializes to a null window, and our loop invariant (“the running best is a fail-soft report of the fold so far”) follows the same shape. *Formal Verification of Minimax Algorithms* [5] verifies minimax variants with alpha-beta and transposition tables in Dafny — the closest prior treatment of the table invariant (our TableOK). Knuth and Moore’s classical analysis [4] and the MTD(f) framework [8] supply the algorithmic background; futility pruning in the form sunfish uses traces to Heinz [3].

Our position: to our knowledge this is the first Lean 4 formalization of null-window/MTD-bi search, and the first line of work to pair machine proofs of search soundness with ELO-measured prices on a production engine — one that plays rated games in public. We found no prior formal treatment of king-capture (pseudo-legal) search, of a stalemate correction, of killer-table invariants, or of a machine-checked refutation of transposition-table point-consistency under gamma-adaptive Late Move Reductions (Cex. 7.4).

11 Lessons and Threats to Validity

The model-code gap. All theorems are about the Lean model of Section 2, not about `sunfish.py`. The gap is managed, not closed: the model is line-mapped (Figure 1; the correspondence table in `formal/README.md` pairs every Lean definition with the sunfish lines it transcribes, audited at commit `9b1a7b4` and re-audited at `7fdd741`), the counterexamples are constructed to exercise exactly the modeled mechanism, and features excluded from a given model are excluded *explicitly* with a stated reason (Section 2.4). Still, quiescence is collapsed into `eval`, move ordering is modeled by its load-bearing consequence only, and no chess rules are formalized at all. A reader should trust the theorems as statements about the mechanisms, quantified over all abstract games, not as end-to-end verification of the Python.

Two author claims refuted by the proof assistant, mid-project. We record these because they are the strongest evidence in this paper that the machine was doing work the authors could not do by inspection. (1) *The fail-high-vs- V_{full} error.* While specifying LMR we claimed that fail-high reports remain sound against full negamax — the re-search guard, we reasoned, ensures no reduced fail high is ever yielded. Lean refused the induction: the guard protects only the *immediately* reduced move, while a parent fail high is certified by child fail lows, which are only sound against the children’s *reduced* values — the contamination is recursive and cannot be scrubbed at one level. This is why the interval-shaped statement of Section 7 had to make V^{hi} ’s children V^{lo} -valued, and why the mutually recursive pair existed at all — before both were deleted as guaranteeing nothing. (2) *The V_{reduced} target error.* We claimed the folklore fail-low target — reducible moves simply valued one ply shallower (negamaxShallow; deleted along with the interval-shaped statements it fed) — was the right lower target. Also false: on the re-search fall-through, the yielded value bounds the *deep* child value while the shallow value has just failed high and can exceed everything yielded; the sound per-move entry is the *minimum* of the two depths. Both errors would have survived code review; neither survived `lake build`.

Measurement contamination. Every rule in `docs/TESTING.md` exists because its absence produced a confident wrong number in this repository: the -60 ELO phantom regression (shared checkout), the $\pm$ dozens swings from cross-game table persistence (process reuse), fictional error bars from deterministic engines without opening books, and the long-TC time bug invisible at fast TC. We treat the methodology document as part of the scientific apparatus, on the same footing as the proofs.

Threats. Single engine, single day, one maintainer’s conjectures; ELO error bars at 300 games are ± 30 -ish and several middle-block numbers in Table 3 are single runs; the taxonomy of Section 7 is validated on one engine family (null-window MTD-bi) and its transfer to PVS/aspiration engines is argued, not proven.

12 Availability

Everything is in the public sunfish repository [1]: the engine (`sunfish.py`), the Lean development (`formal/`, building with `lake build` from a pinned Lean 4.12.0 toolchain, zero dependencies beyond core Lean and the `omega` tactic — no Mathlib — so a full build takes seconds), the methodology (`docs/TESTING.md`), and the lemma inventory, correspondence tables and PR-guideline ledger (`formal/README.md`). CI builds the proofs and plays real protocol games: fastchess tournaments and a mock-Lichess integration test that drives the engine over the wire exactly as the production bot is driven. The engine itself plays continuously on Lichess as `sunfish-engine`.

Acknowledgments.

References

- [1] Thomas Dybdahl Ahle and contributors. Sunfish: a python chess engine in 131 lines of code. <https://github.com/thomasahle/sunfish>, 2026. Formal development in `formal/`; plays on Lichess as `sunfish-engine`.
- [2] Disservin and contributors. fastchess: a command-line tool for managing chess engine tournaments. <https://github.com/Disservin/fastchess>, 2025.
- [3] Ernst A. Heinz. Extended futility pruning. *ICCA Journal*, 21(2):75–83, 1998.
- [4] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi:10.1016/0004-3702(75)90019-3.
- [5] Formal verification of minimax algorithms. arXiv preprint, 2025. Verifies minimax variants with alpha-beta pruning and transposition tables in Dafny. <https://arxiv.org/abs/2509.20138>. TODO: fill in the author list from the arXiv page. [arXiv:2509.20138](https://arxiv.org/abs/2509.20138).
- [6] Tobias Nipkow. Alpha-beta pruning. *Archive of Formal Proofs*, 2024. https://isa-afp.org/entries/Alpha_Beta_Pruning.html, Formal proof development.

- [7] Tobias Nipkow. Alpha-beta pruning verified. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPICs*, pages 1:1–1:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. Invited talk. [doi:10.4230/LIPICs.ITP.2024.1](https://doi.org/10.4230/LIPICs.ITP.2024.1).
- [8] Aske Plaat, Jonathan Schaeffer, Wim Pijls, and Arie de Bruin. Best-first fixed-depth minimax algorithms. *Artificial Intelligence*, 87(1–2):255–293, 1996. [doi:10.1016/0004-3702\(95\)00126-3](https://doi.org/10.1016/0004-3702(95)00126-3).